

AWS Deployment Checklist

Everything to confirm before, during, and after taking the AFS Financial CRM live on AWS. For the full step-by-step walkthrough from a bare EC2 instance, see the companion **AWS Deployment Documentation**.

This is a reference checklist, not an interactive tracker — print it or copy it into your deployment ticket and check items off there. Items marked **BLOCKING** will stop the app from starting or leave a real security gap if skipped.

01 Environment variables

Backend reads these from `.env.production`; copy from `.env.example` and fill in every value below. The app refuses to boot in production if any Blocking item is missing or left at its placeholder.

- | | | |
|--------------------------|--|---|
| ☐ | <code>APP_ENV=production</code> ,
<code>DEBUG=false</code> BLOCKING | Boot-time validator rejects <code>DEBUG=true</code> in production. |
| ☐ | <code>JWT_SECRET_KEY</code> — real, random, ≥ 32 characters BLOCKING | Boot-time validator rejects the shipped placeholder or anything under 32 chars. |
| ☐ | <code>ENCRYPTION_KEY</code> — real, random, ≥ 32 characters, different from <code>JWT_SECRET_KEY</code> BLOCKING | Used for at-rest field encryption (integration credentials). Generate with <code>openssl rand -base64 32</code> . |
| ☐ | <code>MONGO_URI</code> ,
<code>MONGO_DB_NAME</code> | Must include <code>?replicaSet=rs0</code> — the app uses multi-document transactions. |
| ☐ | <code>REDIS_URL</code> | Sessions, OTPs, rate limiting, and lockouts all live here. |
| ☐ | <code>OTP_MODE=production</code> | Never leave as <code>development</code> in production — that mode uses a fixed, well-known OTP. |

- ☐ **AWS S3:** `AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY` , `AWS_REGION` , `S3_BUCKET_NAME` Prefer an EC2 instance role over long-lived keys where possible — see Security Groups section.
- ☐ `CORS_ORIGINS` Real production frontend origin(s) only — comma-separated, no wildcard.
- ☐ `FRONTEND_BASE_URL` Real production URL — used to build the Meta OAuth callback redirect. Never localhost.
- ☐ `TRUSTED_PROXY_IPS` The reverse proxy's own address — `127.0.0.1` if Nginx runs on this same instance. Never a wildcard.
- ☐ Rate limiting, OTP, logout, secure-link expiry tunables Sane defaults are already set; confirm they match your actual policy before launch.

02 Production configuration

- ☐ App refuses to boot with an insecure config **BLOCKING** Confirm by starting the app once with a deliberately bad `.env.production` (placeholder secret) and watching it fail fast, not silently serve traffic.
- ☐ No `.env*` file committed to version control with real secrets
- ☐ MongoDB running as a replica set (even single-node)
- ☐ Redis persistence/eviction policy reviewed Rate-limit windows and account lockouts live in Redis only — an eviction policy that's too aggressive clears them silently.

03 FastAPI backend

- ☐ Installed into a virtualenv from `backend/pyproject.toml`
- ☐ Served via Uvicorn behind Nginx Not exposed directly to the internet on port 8000.
- ☐ Arq worker process running continuously **BLOCKING** Separate from the API process — `arq` `app.worker.worker_settings.WorkerSettings` . Meta token refresh, reminders, communication, and referral-partner background tasks all depend on it. Easy to forget since nothing fails loudly if it's not running.
- ☐ `GET /api/v1/health` returns `{"status": "ok"}`

04 React production build

- ☐ `npm run build` from `frontend/` — outputs static files to `frontend/dist`
- ☐ Built with `frontend/.env.production` (if any `VITE_*` overrides are needed) Not required by default — the app's own relative `/api/v1` base path works as long as Nginx proxies `/api/` to the backend on the same origin.
- ☐ Served as static files by Nginx, not by `vite dev` / `vite preview`

05 MongoDB

- ☐ Single-node replica set initialized (`rs.initiate()`)
- ☐ Bound to localhost only, not `0.0.0.0` , unless using a managed service (Atlas/DocumentDB) with its own network controls
- ☐ Indexes created on app startup Happens automatically via the app's own lifespan hooks — confirm no errors in the startup log.
- ☐ Backup strategy in place See Backup Strategy below.

06 Redis

- ☐ Bound to localhost only, not `0.0.0.0` , unless using a managed service (ElastiCache) with its own network controls
- ☐ A password/AUTH set if reachable from anywhere other than localhost

07 Nginx configuration

- ☐ Serves `frontend/dist` as static files with SPA fallback (`try_files $uri $uri/ /index.html`)
- ☐ Proxies `/api/` to the backend and forwards `X-Forwarded-For` + `X-Forwarded-Proto` **BLOCKING** Without these, the backend's proxy-trust middleware can't recover the real client IP — rate limiting and audit-log IPs silently collapse onto Nginx's own address.
- ☐ Client body size limit raised for document uploads

- ☐ Gzip/Brotli compression enabled for static assets

08 SSL (Let's Encrypt)

- ☐ Certificate issued via Certbot for the real domain
- ☐ HTTP → HTTPS redirect configured
- ☐ Auto-renewal confirmed working (`certbot renew --dry-run`)

09 Domain & DNS

- ☐ A/AAAA record pointing at the instance's Elastic IP
- ☐ `FRONTEND_BASE_URL` and `CORS_ORIGINS` match the real domain, not an IP or localhost

10 Firewall & Security Groups

- ☐ Security Group: inbound 22 (SSH, restricted to a known IP), 80, 443 only
- ☐ MongoDB (27017) and Redis (6379) NOT open to the internet
- ☐ Backend port 8000 NOT open to the internet — reached only via Nginx on localhost
- ☐ UFW (or equivalent host firewall) enabled as a second layer

11 Process management (systemd)

- ☐ Backend API service — enabled, auto-restarts on failure
- ☐ Arq worker service — enabled, auto-restarts on failure, separate from the API service
- ☐ Both enabled for auto-start on reboot (`systemctl enable`)

12 Logging

- ☐ Application logs written to a persistent location, not just journald's default retention
- ☐ Nginx access/error logs rotated (logrotate)
- ☐ No secrets/PII in logs The Meta OAuth token-logging bug found this pass is fixed — confirm no similar diagnostic logging was added since.

13 Backup strategy

- ☐ MongoDB: scheduled `mongodump` (or EBS snapshot) with a tested restore procedure
- ☐ S3 documents: versioning enabled on the bucket
- ☐ Backups stored off-instance (S3, not the same EBS volume)

14 Monitoring

- ☐ Uptime check against `GET /api/v1/health`
- ☐ Disk space alert on the EC2 volume (Mongo/logs can fill it)
- ☐ CPU/memory alarms via CloudWatch

15 Health checks

- ☐ `/api/v1/health` checked by systemd (or an external monitor) to auto-restart on failure

16 Rollback plan

- ☐ Previous release's git commit/tag recorded before deploying
- ☐ Rollback = checkout previous commit, reinstall deps, rebuild frontend, restart services — documented and rehearsed once
- ☐ Database migrations (if any) are reviewed for reversibility before deploying

17 Post-deployment verification

Run through this in a real browser against the live URL after every deploy.

- ☐ Owner login works
 - ☐ A test lead can be created, assigned, and viewed
 - ☐ A secure application link can be generated, opened, and completed end-to-end
 - ☐ Document upload succeeds and the file is reachable in S3
 - ☐ A real Meta webhook test event is received and produces a lead
 - ☐ HTTPS padlock is valid, HTTP redirects to HTTPS
 - ☐ Non-Phase-1 nav items show "Soon" and their direct URLs resolve to Coming Soon, not the real page
 - ☐ Formal load/penetration test scheduled against staging before wider rollout
- Not something this audit could reproduce — see the Readiness Report's "Not Verifiable Here" section.

18 Reference: required environment variables

VARIABLE	EXAMPLE / NOTE
APP_ENV	production
DEBUG	false
JWT_SECRET_KEY	32+ random characters
ENCRYPTION_KEY	32+ random characters, different from above
MONGO_URI	mongodb://localhost:27017/?replicaSet=rs0
MONGO_DB_NAME	afs_crm
REDIS_URL	redis://localhost:6379/0
OTP_MODE	production
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	or an instance role
AWS_REGION	ap-south-1 (or your region)
S3_BUCKET_NAME	real bucket name

CORS_ORIGINS	https://your-domain.com
FRONTEND_BASE_URL	https://your-domain.com
TRUSTED_PROXY_IPS	127.0.0.1
RATE_LIMIT_PER_MINUTE	60 (default)

AFS Financial CRM — AWS Deployment Checklist · see the companion AWS Deployment Documentation for the full step-by-step walkthrough.